GenotypeQuant: using microsatellites to estimate the number of different genotypes from a single species in an eDNA sample

Filipe Alberto & Lauran Liggan

2024-06-12

Background

The original motivation to this analysis was to estimate the number of kelp gametophytes (microscopic haploid stage of kelps) present in an eDNA sample by using microsatellite genotyping.

Traditionally, e-DNA barcode methods estimate species presence-absence and abundance but do not estimate the number of individual genotypes present in a sample for a given target species. We devised a numerical maximum likelihood (NMLE) approach to estimate the number of genotypes most likely to have produced the observed counts of unique microsatellite alleles amplified from e-DNA at different loci. The method is akin to forensics analysis of crime scenes with multiple suspects. The caveat is that the number of genotypes in a eDNA sample can be much larger than the number of suspects in a crime scene rendering analytic maximum likelihood approaches challenging. Our numerical algorithm only requires the reference population allele frequencies and a vector with the counts of unique alleles amplified at each locus from the eDNA sample. Correction for deviation from Hardy-Weinberg equilibrium are available. The NMLE has both high precision and accuracy with a relatively small number of loci used (see precision and number of markers and alleles analysis *TODO*).

Input file: microsatellite genotypes for the reference population

The input file containing the reference population microsatellite allele codes is a data.frame with one column per locus (no separator between the two alleles) one row per individual. Column names are used for locus names but there are no individual (row) codes, i.e. the first column is the information for the first locus. Use the same number of digits to code each allele. For example, 120126 is a possible value for a diploid coded with three digits per allele. At this stage, the reference data needs to be in diploid format. Missing data needs to be coded with zeros, e.g., 0, 00, 0000, or 000000.

```
# Reading a matrix with the reference population
# microsatellite alleles; one locus per column, and 3-digit
# per allele
genotypes.ref.pop <- read.delim("../Ref_Pop_Msat_All.txt")
```

The first 10 lines of the input data are shown in table 1. Note that the package does not come with an example data set (FINAL VERSION WILL).

Table 1: The first ten lines of the genotype table for the reference population. Note that here we are using 3 digit codes for alleles.

Ner13	Ner14	Ner19	Ner2	Ner11	Ner4	Ner6
195203	193201	193199	262262	289310	280294	165165
201203	195204	193197	262272	0	280294	160165
199203	201227	199213	262262	289301	280299	162165
199199	195204	193193	260262	343347	268295	162165

Ner13	Ner14	Ner19	Ner2	Ner11	Ner4	Ner6
211221	193204	193195	272272	285293	295295	160174
196213	195201	191193	262272	0	292299	0
199199	198210	193195	272272	0	295295	160168
199215	204233	195218	266266	285351	295302	160160
195213	193193	191191	262262	285310	280295	162162
199213	204215	191195	262272	0	295295	165165

Creating distributions with the counts of unique alleles sampled from 1 to g genotypes

The GenotypeQuant package can be installed from GitHub

```
# We need package devtools to install from GitHub
library(devtools)
install_github("UWMAlberto-Lab/GenotypeQuant")
```

We can now load the package

```
# Loading the package
library(GenotypeQuant)
```

Next, we use function *MakeAllNumberDist* to generate sampling distributions of number of unique alleles for different number of genotypes (given the reference population allele frequencies).

```
# Creating the distributions of counts of unique allele for each loci and number of genotypes
All.Num.dist<-MakeAllNumberDist(genotypes.df=genotypes.ref.pop,ngametos=80,All.nchar=3,
                        Fis=FALSE,Fis.treshold=0.1,npermutations=5000)
```

The first argument *genotypes.df* takes the input data for the reference population (see above). The argument *ngametos* sets the maximum number of haploid genotypes *g* used to sample the reference population allele frequencies. You will have different distributions from 1 to g genes for each locus, and each distribution will have *npermutations* elements. When running your analysis with observed data you can not estimate a higher number of genotypes than your *ngametos* value. For diploid genotypes multiply the parameter by 2, i.e. if you are interested in estimating up to 40 genotypes set this parameter to at least 80. Also, to visualize the maximum likelihood curve and confidence intervals *ngametos* should go above the expected number of genotypes. Remember you can always run the function again with a larger *ngametos* value (generating the distributions takes a few minutes). Distributions of unique allele counts will be created for each type of sample from 1 to *ngametos* genes and for each locus. If your system is characterized by significant homozygosity excess (e.g., selfing), you can use an Fis correction. Genepop will be used to calculate Qintra, the observed frequencies of identical pairs of genes in the reference population. Alleles are then sampled sequentially with probability Qintra to sample the previously sampled allele and 1-Qintra to sample any other allele according to their allelic frequencies. When applying Fis correction it is recommended to only correct for Fis values above a certain threshold. The argument *FIs.threshold* can be used for this and defaults to 0.1, while the *Fis* argument defaults to FALSE

Below, we show a subset for two distributions for all locus. Shown are the first 20 values of the distributions of counts of unique alleles sampled by 5 and 15 genes, in tables 2 and 3 respectively.

Table 2: The first twenty values in the distribution of counts of unique alleles (columns), for the seven locus (rows), sampled for the case of eDNA samples with 5 genes

Ner13	4	4	4	4	4	5	5	4	4	5	4	4	5	5	4	4	2	3	4	5
Ner14	3	4	4	5	5	4	4	5	3	3	4	3	3	4	3	5	4	5	1	3
Ner19	4	3	4	4	2	3	3	2	3	4	3	3	2	3	3	4	2	3	2	3
Ner2	3	3	4	3	4	3	4	2	3	3	2	3	5	3	3	2	3	3	3	3
Ner11	2	4	4	4	3	4	4	4	3	4	3	5	3	5	5	4	4	4	4	4
Ner4	2	3	5	5	4	5	4	4	4	5	4	5	5	5	5	4	4	4	4	3
Ner6	2	3	3	3	3	2	1	3	4	2	3	2	1	3	2	3	2	2	4	3

Table 3: The first twenty values in the distribution of counts of unique alleles (columns), for the seven locus (rows), sampled for the case of eDNA samples with 15 genes

Ner13	10	10	9	8	8	9	9	7	8	10	9	10	9	10	10	10	7	7	7	9
Ner14	6	9	9	6	7	7	4	7	10	9	8	8	8	8	7	9	8	9	6	9
Ner19	4	5	5	5	4	5	4	3	4	4	6	3	5	4	5	3	6	5	4	3
Ner2	3	4	4	4	3	4	4	4	5	4	4	4	4	3	4	2	3	3	4	4
Ner11	12	10	10	11	7	9	9	8	10	8	8	9	7	10	10	10	7	10	9	10
Ner4	10	10	8	10	8	7	8	9	5	8	10	6	8	9	6	9	8	9	8	6
Ner6	7	7	3	5	4	3	5	4	4	5	5	5	4	4	4	4	4	4	4	6

Estimating the number of genotypes in the sample

Once we have the allele number distributions created, we can proceed to estimate the number of genotypes that most likely produced an observed count of unique alleles obtained by genotyping an eDNA sample at the same microsatellite loci. At this point, the only thing we need is a vector with the counts for unique alleles genotyped for each locus in the eDNA sample.

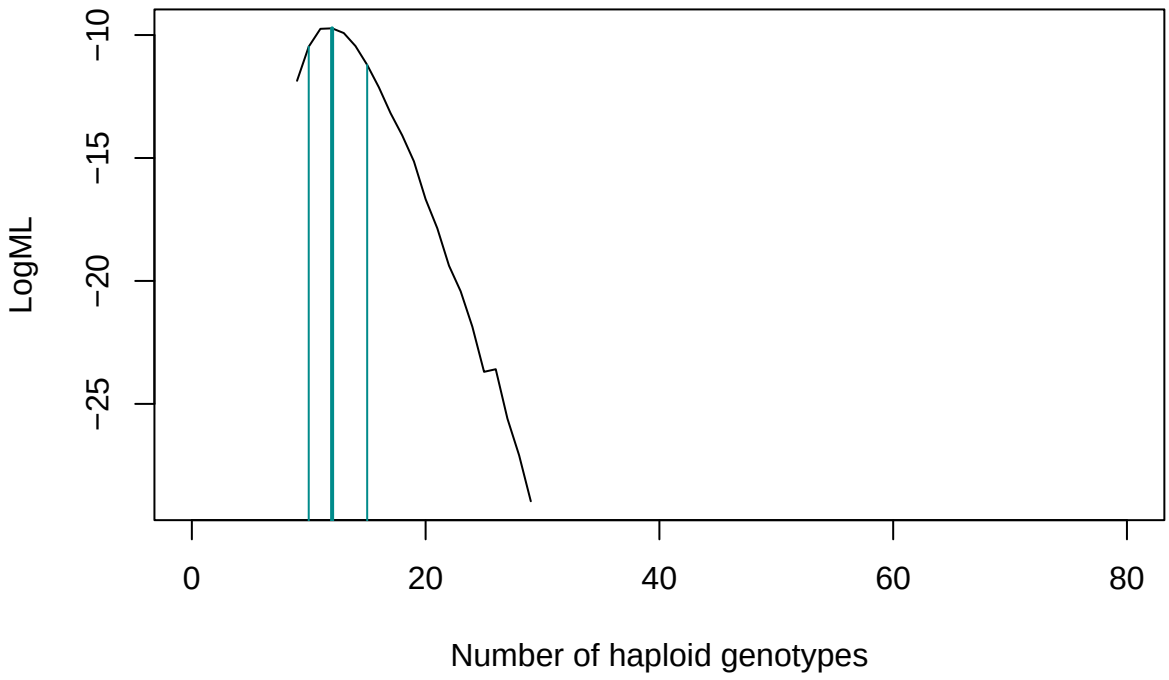
We will actually simulate this vector of observed unique allele counts, while normally you would be using real eDNA genotyping data.

```
# We  simulate what an observed sample created created by 15 haploid genotypes could
# look like
gobs<-15
All.Numb.obs<-apply(All.Num.dist[gobs,,],1, function(x)sample(x,1))
All.Numb.obs
```

```
## [1] 7 6 5 3 9 6 5
```

Now we can use this simulated observed data to run the algorithm. We are estimating the number of haploid genotypes here (kelp gametophytes). You could change the argument *ploidy* to 2 for estimating the number of diploid genotypes. You need to use the same loci order in this vector and reference population genotype table.

```
Estimate.G <- G.Estimate(Obs.N.all = All.Numb.obs, All.Num.Dist = All.Num.dist,
    ploidy = 1)
```




```
Estimate.G
```

```
## $estimateM
##      G.hat lower.95.CI upper.95.CI
## [1,]    12          10          15
##
## $LogML
##  [1]       -Inf       -Inf       -Inf       -Inf       -Inf       -Inf
##  [7]       -Inf       -Inf -11.860037 -10.472847  -9.747174  -9.728406
## [13]  -9.919510 -10.448075 -11.216380 -12.125613 -13.176075 -14.083329
## [19] -15.139211 -16.675379 -17.860815 -19.379017 -20.423129 -21.864680
## [25] -23.695967 -23.590627 -25.601193 -27.092279 -28.958652       -Inf
## [31]       -Inf       -Inf       -Inf       -Inf       -Inf       -Inf
## [37]       -Inf       -Inf       -Inf       -Inf       -Inf       -Inf
## [43]       -Inf       -Inf       -Inf       -Inf       -Inf       -Inf
## [49]       -Inf       -Inf       -Inf       -Inf       -Inf       -Inf
## [55]       -Inf       -Inf       -Inf       -Inf       -Inf       -Inf
## [61]       -Inf       -Inf       -Inf       -Inf       -Inf       -Inf
## [67]       -Inf       -Inf       -Inf       -Inf       -Inf       -Inf
## [73]       -Inf       -Inf       -Inf       -Inf       -Inf       -Inf
## [79]       -Inf       -Inf
##
## $Amp.Qual
##           L.1      L.2       L.3        L.4        L.5       L.6       L.7
## [1,] 0.451267 0.180799 0.7655675 0.06082799 0.09314225 0.2034394 0.6278378
##         ChiSq   P.value
## [1,] 2.382881 0.7940207
```

The function *G.Estimate* produced a log maximum-likelihood plot with vertical lines showing the estimated number of haploid genotypes that could have generated the counts of unique alleles observed while genotyping an eDNA sample and the respective 95% confidence interval bounds. The function also returns a list with components *estimateM* with the values for estimated G and 95% CI bounds, *logML* with the log-likelihood values for the different values of G, and *AmpQual* with the results of a chi-square test comparing the observed frequencies of alleles amplified in the eDNA sample with those expected given the reference population (see *detecting amplification problems* below).

Running more than one sample

Often there will be more than one sample from which to estimate the number of genotypes Below we show a file with 20 samples (rows), each with observed (genotyped) counts of unique alleles for each of the seven loci. The only column that you would not have in a real file is the last one; here again we simulated the observed data and therefore know the *real* number of haploid genotypes. This provides a good opportunity to evaluate the precision of the method.

```
ObservedData<-read.delim("Observed samples.txt")
```

Table 4: Example of an input file (ObservedData) with observed data for 20 eDNA samples for the same seven loci

Ner13	Ner14	Ner19	Ner2	Ner11	Ner4	Ner6	G.obs
9	7	3	4	8	8	4	13
8	8	6	4	9	10	5	18
6	4	3	4	7	5	3	8
8	5	4	4	11	7	3	15
4	6	6	3	5	5	3	8
9	7	4	1	8	9	5	14
4	4	4	3	4	5	1	5
8	6	6	4	7	10	3	14
6	5	4	4	6	6	5	8
10	7	5	4	11	9	4	25
13	6	4	4	13	9	6	21
6	4	3	3	5	6	4	6
10	7	5	4	13	9	5	18
12	9	6	5	11	10	6	24
9	5	5	4	9	6	6	13
6	7	3	4	6	6	4	8
10	8	6	3	12	7	7	18
11	10	5	5	11	11	3	22
8	6	6	2	5	7	4	14
6	7	3	3	7	5	2	8

Finally, we apply a loop to run function *G.Estimate* over each of the observed samples. Then we store the results in a new object and combine them with the observed data for a final output file. Note we turn the plotting functionality off to simplify the script. You could edit the script using pdf() to export the plots for all 20 samples to an external file.

```
# A data.frame to store the results
results.G.DF <- data.frame(matrix(nrow = 20, ncol = 3))
names(results.G.DF) <- c("G.hat", "lower.95.CI", "upper.95.CI")

for (s in 1:nrow(ObservedData)) {
    results.G.DF[s, ] <- G.Estimate(Obs.N.all = as.numeric(ObservedData[s,
```

```
        1:7]), All.Num.Dist = All.Num.dist, ploidy = 1, ML.curve = F)$estimateM
    # note the column indexation (1:7), to remove the last
    # column with the 'true' number of gametophytes
}
# Finally we bind the results with the observed data
Output <- cbind(ObservedData, results.G.DF)
```

Table 5: Binding the estimation results (last three columns) to the observed data. The first seven columns have the alleles observed at each locus, then the G.obs has the 'real' number of haploid genotypes (not known usually), G.hat is the estimated number of haploid genotypes using this data, the lower and upper bounds of the 95% confindence interval are given in the last two colums.

Ner13	Ner14	Ner19	Ner2	Ner11	Ner4	Ner6	G.obs	G.hat	lower.95.CI	upper.95.CI
9	7	3	4	8	8	4	13	13	11	17
8	8	6	4	9	10	5	18	17	14	23
6	4	3	4	7	5	3	8	8	7	10
8	5	4	4	11	7	3	15	13	11	17
4	6	6	3	5	5	3	8	7	6	9
9	7	4	1	8	9	5	14	13	11	17
4	4	4	3	4	5	1	5	5	5	6
8	6	6	4	7	10	3	14	14	11	17
6	5	4	4	6	6	5	8	9	7	11
10	7	5	4	11	9	4	25	18	14	24
13	6	4	4	13	9	6	21	24	18	30
6	4	3	3	5	6	4	6	7	6	9
10	7	5	4	13	9	5	18	20	16	28
12	9	6	5	11	10	6	24	27	20	35
9	5	5	4	9	6	6	13	13	11	17
6	7	3	4	6	6	4	8	9	8	12
10	8	6	3	12	7	7	18	20	16	26
11	10	5	5	11	11	3	22	23	18	32
8	6	6	2	5	7	4	14	10	8	12
6	7	3	3	7	5	2	8	8	7	10

For this particular reference population allele frequency table, with only 7 loci and moderate allelic diversity, the 95% confidence intervals are not too wide for 10 or less genes sampled. The precision can be improved by using more loci or more polymorphic loci or both (see vignette on precision and number of loci and alleles analysis TODO).

Detecting amplification problems

An eDNA sample might fail to amplify all the microsatellite alleles present. More concerning, for problematic samples, there might be heterogeneity across loci in the number of alleles that are amplified. This can result in observed allele counts across loci that do not follow the different allele richness of each loci in the reference population. For example, a marker with more alleles in the reference population might have fewer amplified alleles in the eDNA sample than a loci with fewer alleles in the reference population. This will result in a biased low estimate of the number of genotypes present and non-informative confidence intervals. We used a simple chi-square test to measure how the observed number of alleles amplified compares with the expected number of alleles, given the diversity of the different loci in the reference population and the total number of alleles amplified. The chi-square test measures the quality of the eDNA sample. It provides complementary

information to the CI because a lower allele richness, number of loci used, or both can also result in wide CI for eDNA samples with high quality (i.e., without amplification problems). The statistic ((0-E)^2)/E is given for each locus to allow detection of which loci deviate more from expectations (i.e., the loci where some alleles might have not amplified); the sum of these values, the Chi-square statistic, is given as well as the p-value with degrees of freedom equal to the number of loci minus two.

Below, as an example, we again generate a possible vector of observed alleles by sampling the reference population for a possible outcome with number of genotypes G=15, and visualize again the *G.estimate* function output. We then replace the number of alleles for the first loci, 7, by 2, a value much lower than expected for sampling this loci from the reference population for G=15. We leave the number of alleles found for other loci untouched. Note how the ((0-E)^2)/E for this loci increases from 0.85 to 5.25, as well as the overall chi-square statistic from 3.44 to 9.38, while the p-value decreases from 0.63 to 0.09. Also note how the G estimate decreased from 14 to 9.

```
# We simulate what an observed sample created created by 15 haploid
# genotypes could look like
gobs <- 15
All.Numb.obs <- apply(All.Num.dist[gobs, , ], 1, function(x) sample(x,
    1))
All.Numb.obs
```

```
## [1] 7 8 5 5 8 7 5
```

```
Estimate.G <- G.Estimate(Obs.N.all = All.Numb.obs, All.Num.Dist = All.Num.dist,
    , ML.curve = FALSE, ploidy = 1)
Estimate.G$estimateM
```

```
##      G.hat lower.95.CI upper.95.CI
## [1,]    14          11          18
```

```
Estimate.G$Amp.Qual
```

```
##            L.1         L.2       L.3      L.4        L.5       L.6       L.7
## [1,] 0.8479692 0.003602366 0.4405448 1.609461 0.09546842 0.1057225 0.3384282
##         ChiSq   P.value
## [1,] 3.441196 0.6323035
```

```
# replacing the number of observed alleles in Loci 1 by a lower
# number
All.Numb.obs[1] <- 2
Estimate.G <- G.Estimate(Obs.N.all = All.Numb.obs, All.Num.Dist = All.Num.dist,
    ML.curve = FALSE, ploidy = 1)
Estimate.G$estimateM
```

```
##      G.hat lower.95.CI upper.95.CI
## [1,]     9           9           9
```

```
Estimate.G$Amp.Qual
```

```
##           L.1       L.2       L.3      L.4          L.5         L.6       L.7
## [1,] 5.251973 0.1548233 0.8673131 2.386305 0.0005911994 0.000177452 0.7199493
##         ChiSq    P.value
## [1,] 9.381133 0.09479417
```